

Rishu Python Reporting (Pro Level FastAPI Project)

This is a production-style FastAPI reporting system with clean structure, validation, and routing logic. Fully optimized to run in Termux without errors.

Termux Setup (Step-by-Step):

1. pkg update && pkg upgrade -y
2. pkg install python -y
3. pip install fastapi uvicorn pydantic
4. Create file: main.py
5. Paste below code
6. Run: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
7. Open: <http://127.0.0.1:8000/docs>

```
# =====
# Rishu Python Reporting API (PRO)
# =====

from datetime import datetime
from typing import Dict, Any, Optional
import uuid

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(
    title="Rishu Reporting API",
    description="Pro-level moderation backend system",
    version="2.0.0"
)

# In-memory DB (replace with real DB in future)
REPORTS_DATABASE: Dict[str, Dict[str, Any]] = {}

# Request Model
class ReportRequest(BaseModel):
    target_object_id: str = Field(..., min_length=3)
    reporter_id: str = Field(..., min_length=3)
    primary_category: str = Field(..., min_length=3)
    sub_category: Optional[str] = "general"

# Response Model
class ReportResponse(BaseModel):
    status: str
    ticket_reference: str
    assigned_queue: str
    timestamp: str

# Moderation Engine
class ModerationSystemEngine:
    HIGH_RISK = {"hate_speech", "violence", "self_harm", "harassment"}

    @staticmethod
    def route(category: str) -> str:
        if category.lower() in ModerationSystemEngine.HIGH_RISK:
            return "HUMAN_MODERATION_QUEUE"
        return "AUTO_FILTER_QUEUE"

# Submit Report
@app.post("/api/v1/report", response_model=ReportResponse)
def submit_report(payload: ReportRequest):
    case_id = f"CASE-{uuid.uuid4().hex[:10].upper()}"
    timestamp = datetime.utcnow().isoformat()
    queue = ModerationSystemEngine.route(payload.primary_category)
    REPORTS_DATABASE[case_id] = {
```

```

        "case_id": case_id,
        "target": payload.target_object_id,
        "reporter": payload.reporter_id,
        "category": payload.primary_category,
        "sub_category": payload.sub_category,
        "queue": queue,
        "status": "Pending",
        "created_at": timestamp
    }

    return {
        "status": "success",
        "ticket_reference": case_id,
        "assigned_queue": queue,
        "timestamp": timestamp
    }

# Get Report Status
@app.get("/api/v1/report/{case_id}")
def get_report(case_id: str):

    report = REPORTS_DATABASE.get(case_id)

    if not report:
        raise HTTPException(status_code=404, detail="Report not found")

    return report

# Health Check
@app.get("/")
def health():
    return {"message": "API Running Successfully 🟢"}

```

Next Upgrade Ideas:

- Add MongoDB / SQLite
- Add Login/Auth System
- Deploy on Render / Railway
- Connect with Frontend (React / Flutter)